Problem A:

Problem A: A History of Computing

1. (1936):
In 1936 Alan Turing published his paper "On Computable Numbers, with an Application to the Entscheidungsproblem", in wich he introduces the theoretical model of the "Turing Machine".

2. (1947):
At Bell Labs John Bardeen, Walter Brattain and William Shockley invented solid-state transistor, building block of all computers

3. (1958):
In 1958 Physicist William Higinbotham created first video game, Tennis for Two, on an oscilloscope

4. (1969):
In 1969 at Bell Labs researchers Ken Thompson, Dennis Ritchie created the Unix operating system, Unix is foundation for Linux, macOS, Ios and etc.

5. (1981):
In 1981 IBM released the Model 5150, wich became standard for the next decades and introduced architecture (x86).

6. (1991):
In 1991 the World Wide Web and the first web page launced by Tim Berners-Lee.

Problem B:

Problem B: Pixel Art

1. Checkerboard

The top-left cell is always 0.

0 1 0 1
1 0 1 0
0 1 0 1
1 0 1 0

x, y coordinate

(0,0) (0,1) (0,2) (0,3)
(1,0) (1,1) (1,2) (1,3)
(2,0) (2,1) (2,2) (2,3)
(3,0) (3,1) (3,2) (3,3)

We can conclude that:

$[x_i, x_j] = 1$ if $(x_i + x_j) \pmod 2$

Time complexity $O(N^2)$
Space complexity $O(N^2)$

1 0 1 0 1

In every unit we move one point vertically or horizontally it changes its state. Using that top left cell is always 0 and parity of the checkboard pattern. If for any cell (x_i, x_j), if the sum $x_i + x_j$ is even its in one state, if its odd its in another state. Knowing that $x_i + x_j$ where $(x_i = 0, x_j = 0)$ the top-left cell is white(0). We can conclude that even are white and odd is black.

2. Diamond Pattern

The diamond pattern is defined by the Manhattan distance (L_1) from the centre of the grid

0 0 1 0 0
0 1 1 1 0
1 1 1 1 1
0 1 1 1 0
0 0 1 0 0

N is always odd
Time complexity $O(N^2)$
Space complexity $O(N^2)$

mid = [n/2]

The center of the grid (x, y) is (mid, mid)

- When N*N grid is N odd the borders hit the distance exactly at manhattan dist.

Manhattan distance for any cell is (x_i, x_j) to the centre is $|x_i - mid| + |x_j - mid| \leq mid$

we can conclude:
$[x_i, x_j] = 1$ is

```python
def generate_shape(n, shape):
    # Creates blank grid NxN using lists
    grid = [[0 for _ in range(n)] for _ in range(n)]

    # Checks if the shape is checkerboard
    if shape == "checkerboard":
        for y in range(n):
            for x in range(n):
                # Checks the parity of the cell
                grid[y][x] = (y + x) % 2

    # Checks if the shape is diamond
    elif shape == "diamond":
        mid = n // 2
        for x in range(n):
            for y in range(n):
                # Checks if equal to Manhattens distance
                if abs(x - mid) + abs(y - mid) <= mid:
                    grid[y][x] = 1

    return grid
```

Problem C:

Problem C: Moore's Law

$$T(t) = T_0 \times 2^{t/2}$$

- T+ is exponential growth function.
- t/2 means every year it doubles every year

(a)

$T_0 = 2300$ starting value

Reference year = 1971

$t = 2026 - 1971 = 55$

$T(55) = 2300 \cdot 2^{55/2}$

$T(55) = 2300 \cdot 2^{27.5} = 2300 \cdot 2^{27} \cdot \sqrt{2} \approx 189,812,531.$

2300

$T(55) = 436,568,821,306 \approx 436,57$ Billion transistor in 2026

(b)

It is not officially reported by the Apple the number of transistors of Apple M5 but it estimated to be around 28-billion transistors based on previous architecture of Apple M4. [illegible]. Also high- and consumer chips released in the 2025-2026 range from 28 billion to 100 billion. Large GPUs like NVIDIA Blackwell 208 billion and NVIDIA Rubin 336 billion.

Conclusion:
Moore's Law overestimates the actual count by ≈93,42 % on lowend and by ≈21,12 % on high-end.

In my opinion the problem with Moore's law is that it oversimplifies the reality of growth of transistors. Also the physical limits of atomic-scale manufacturing (3 nanometers being current-edge standard). Also in my opinion division into CPU and GPU should count the increase in the number of transistors.

Problem D:

Problem D: The Simultaneous Strike

(a)

The game's rules state that all events on the same frame must be applied before checking for a KO.

But in the pseudocode we clearly see that code first checks the KO in line 3 and breaks the cycle is positive in line 4:

```
3    is player1_hp ≤ 0 OR player2_hp ≤ 0 then
4        break
```

And after this it applies the damage of events of that frame on line 6

```
6.   opponent_hp ← opponent_hp - event.attack_value
```

This sequence is directly against the Game's rules.

To fix we need to chang the order of sequence in pseudocode. First apply events then check for KO.

```python
from collections import defaultdict
def processGame(events, H):
    # Creates and dict with default values as list []
    frame_map = defaultdict(list)

    # Goes throug each event
    for player, frame, damage in events:
        frame_map[frame].append(
            (player, damage)
        )  # Appends information to the specific frame

    chrono_sorted_frames = sorted(frame_map.keys())  # Sorts the
frames chronologically
    # Sorting frames chronologically ensures that events happen in
chronological order

    # Sets up initial healt of players
    hp1, hp2 = H, H

    # Goes through each frame
    for frame in chrono_sorted_frames:
        # Firs applies the events happened in that frame
        for player, damage in frame_map[frame]:
            if player == 1:
                hp2 -= damage
            elif player == 2:
                hp1 -= damage
        # Then checks for KO
        if hp1 <= 0 or hp2 <= 0:
            break

    # Returns the values and clamps minimum to 0
    return [max(0, hp1), max(0, hp2)]
```

Problem E:

Problem E: Page Rank in the Age of AI slop

$$PR(p_i) = \frac{1-d}{N} + d \sum_{p_j \in M(p_i)} \frac{PR(p_j)}{L(p_j)}$$

$M(p_i)$ – the set of pages that link to p_i

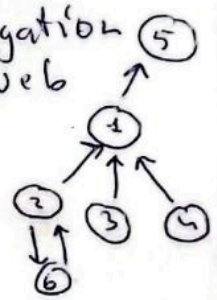
$L(p_j)$ – the number of outbound links from p_j

N – is the total number of pages

d – damping factor ($d = 0.85$)

(a)

In my understanding Page Rank models an Internet-surfer and its navigation in the web. Page Rank views the web as a directed graph where nodes are webpages and edges are ~~links~~ hyperlinks.



- With probability d the surfer picks an outbound link from the current page (p_j) at random. Each links becomes important equal to $\frac{PR(p_j)}{L(p_j)}$ (wich makes it recursive function).
- With probability $1-d$ the surfer goes to the random link in network (N). This makes so the surfer does not get stuck on the one part of graph or group of interconnected links

- Role of d (Dumping Factor)
  - It ensures that the matrix is stochastic which Ensures the Law of conservation of Probability.
    - If a page has no outbound links it will not be Dead End. By adding $\frac{1-d}{N}$ ensures that surfer always has somewhere to go.
  - It ensures that the graph is irreducible.
    - is surfer lands on isolated group of webpages $\frac{1-d}{N}$ ensures it will eventu go the other webpages too.
  - It makes the matrix Aperiodic. which ensures that cyclic oscilations like Page A to page B to page A is broken by random nois introduced by $\frac{1-d}{N}$
- Why it is Recursive?

part $\frac{PR(p_j)}{L(p_j)}$ makes this function Recursive (the function that calls itself). This recursive dependency means we are calculating where the total rank flowing out of a page equals the total rank flowing in

In Page Rank to optimize the algorithms the Power Iteration is used.
Power Iteration assigns equal values to every page then calculates values for each page, takes the new scores and pluck them back ~~until~~ repeat until the scores change very slightly or do not change.

(b)

S_A — Sum of page Rank A (all pages in the AI)

$$S_A = \sum_{p_i \in A} \left(\frac{1-d}{N} + d \sum_{p_j \in M(p_i)} \frac{PR(p_j)}{L(p_j)} \right)$$

$$Sum = \alpha \frac{1-d}{N} = \alpha(1-d) \quad \text{(Teleportations)}$$

$$S_A = \underbrace{\alpha(1-d)}_{\text{Teleportation}} + \underbrace{d\,(\text{Inbou}-d)}_{\text{Links}}$$

Links:
- AI pages only link to AI pages
  $A \to A$
- Human pages link to any random page the probability of $H \to A$ is α

$$S_A = \alpha(1-d) + d(S_A + \alpha S_H)$$

$S_H = (1 - S_A)$

$$S_A = \alpha - \alpha d + d S_A + d\alpha(1 - S_A)$$

$$S_A = \alpha - \alpha d + d S_A + d\alpha - d\alpha S_A$$

$$S_A = \alpha + d S_A - d\alpha S_A$$

$$\alpha = S_A - d S_A + d\alpha S_A$$

$$\alpha = S_A(1 - d + d\alpha)$$

$$S_A = \frac{\alpha}{1 - d + d\alpha}$$

Answer: $S_A = \frac{\alpha}{1 - d + d\alpha}$

(c)

Is pages AI equal to the humans pages ($S_A = S_H$). Since the total is 1

$$S_A = 0.5$$

$$S_A = \frac{\alpha}{1 - d + d\alpha}$$

$$0.5 = \frac{\alpha^*}{1 - d + d\alpha^*}$$

$$0.5 = (1 - d + d\alpha^*) = \alpha^*$$

$$0.5 - 0.5d + 0.5d\alpha^* = \alpha^*$$

$$\alpha^* = \frac{0.5(1-d)}{1 - 0.5d}$$

$$\alpha^* = \frac{1-d}{2-d}$$

$$d = 0.85$$

$$\alpha^* = \frac{1 - 0.85}{2 - 0.85} = \frac{0.15}{1.15} \approx 0.1304$$

Results:

AI pages collectively capture the 50% of the web's authority when they make up ≈13% of the ~~web~~ total (N) web pages.

Conclusion:

This reveals the weakess of PageRank to the AI slop. Because human pages link to ~~the~~ any kind of websites, they may leak rank points to the AI pages. AI pages on the other hand only link to AI they act like a trap as the rank pages do not flow out but only flow in. This one way trap is so essecient it only takes 13% of the Internet to be AI slop so they will be able to trap halfs of Rank scores. This ~~studies~~ proves that link based algorithms are ~~also~~ vulnerable to AI slop and in the future should be replaced by more complex algorithms to prevent AI boom.